CCCS 104 - Data Structures and Algorithms

LEARNING TASK (LINEAR DATA STRUCTURE - LIST)

NAME: **ORIAS, MARK JOSEPH C.** SECTION: ***2A***

RATIONALE

Explain briefly List Linear Data Structure, how it works? What are the common examples? Its applications?
Introduce your develop Python program, what can it do?

A ***List Linear Data Structure*** is a fundamental concept in computer science where elements are arranged sequentially in a specific order. This linear arrangement means each element in the list has a designated predecessor and successor, except for the first and last elements. Lists are designed to store a sequence of zero or more elements, maintaining a defined order that facilitates efficient data storage and manipulation.

The way lists operate depends on their implementation, which can either be ***array-based*** or ***linked-list-based***. In an array-based list, elements are stored in contiguous memory locations, allowing for quick access by index due to the direct memory addressing. However, inserting or deleting elements can be costly, as these operations may require shifting elements to maintain order. In contrast, linked-list-based implementations consist of nodes, where each node contains the data and a reference (or pointer) to the next node in the sequence. This design allows for efficient insertion and deletion at any position without the need for shifting elements. However, accessing elements by index in a linked list is slower compared to arrays, as it requires traversing the list from the beginning to reach the desired position.

Common examples of list linear data structures include arrays and various types of linked lists, such as singly linked lists, doubly linked lists, and circular linked lists. Arrays are fixed-size lists where elements are indexed, making random access very efficient. Singly linked lists allow traversal in a single direction, while doubly linked lists enable

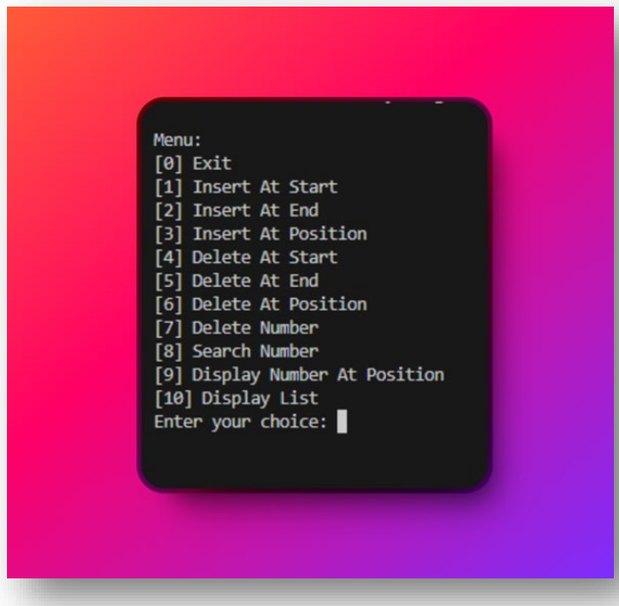
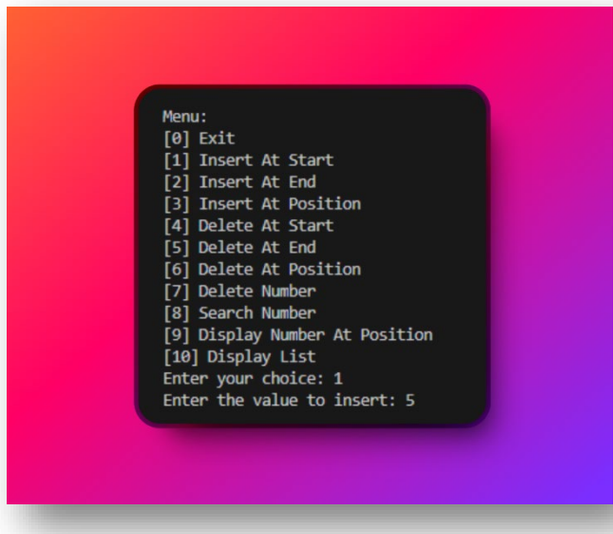
bidirectional traversal through additional pointers that reference both the next and the previous nodes. Circular linked lists, on the other hand, create a loop by having the last node point back to the first, facilitating continuous traversal.

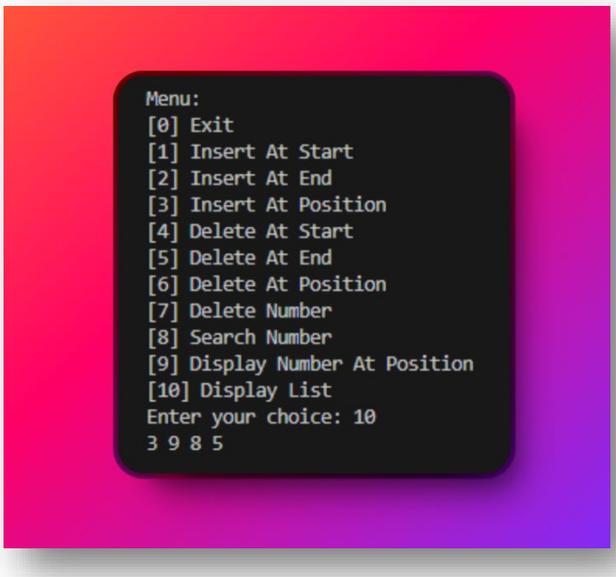
Lists are widely used in numerous ***applications*** due to their flexibility and efficiency in managing ordered data sequences. In compiler design, lists manage symbol tables and syntax parsing, while in operating systems, they handle process scheduling, memory management, and file directory management. In databases, lists are crucial for implementing indexes and managing records, and in graphics, they manage graphical objects and layers. Additionally, lists are integral in statistical analysis packages for organizing datasets and in artificial intelligence for managing state spaces and game trees.

To fully understand how a list linear data structure works, this ***developed python program*** demonstrates the implementation of a List data structure using an array-based approach. The program provides a set of methods to perform common list operations, such as inserting elements at the start, end, or a specific position; deleting elements from the start, end, or a specific position; and searching for and displaying elements. The program also includes a user-friendly menu interface that allows users to interactively perform these operations and manage their list. With built-in error handling, this program ensures robust and effective manipulation of linear data structures in Python.

USER GUIDE

Step by step instructions on how to use your program. Include images for easily visualization

Step 1	Image 1
RUN *the code until the program starts.* *The* ***MENU*** *will pop up or show after doing this step.* *Then, the user is asked to* ***ENTER*** *their* ***CHOICE****.*	
Step 2	**Image 2**
ENTER *the number of operation you want to do with the program.* ***FOR INSTANCE,*** ***CHOICE*** *[1] to [3] will ask the user to enter the* ***VALUE*** *to be* ***INSERTED*** *in the list.* ***CHOICE*** *[4] to [7] will ask the user to enter the* ***VALUE*** *to be DELETED in the list.* ***IF*** *the user enters an* ***INVALID*** *value, (e.g., "a", "$"), the program will print an error*	

message and will ask again for a choice of operation.	
Step 3	***Image 3***
TO CHECK *If the value the user inserted into the list,* ***CHOICE*** *[9] will display the number at position, while;* ***CHOICE*** *[10] will print all the value inserted into the list.* ***WHILE, CHOICE*** *[9] will display the value at a certain position/index.*	
Step 4	***Image 4***
AFTER *the user is done using the program.* ***INPUT*** *"0" to* ***EXIT*** *the program. A confirmation will pop up and will ask (y/n).* *Just type* ***"y"*** *to exit the program, and* ***"n"*** *to continue using the program.*	

PROGRAM CODE

Insert your Python code here, be sure to add comments to describe each line/set of codes…

```python
# insert a value at the start of the list
def insert_at_start(list, value):
    list.insert(0, value)

# insert a value at the end of the list
def insert_at_end(list, value):
    list.append(value)

# insert a value at a specific position in the list
def insert_at_position(list, index, value):
    list.insert(index, value)

# delete the value at the start of the list
def delete_at_start(list):
    if len(list) == 0:
        print("List is empty, cannot delete")
    else:
        list.pop(0)

# delete the value at the end of the list
def delete_at_end(list):
    if len(list) == 0:
        print("List is empty, cannot delete")
    else:
        list.pop()

# delete the value at a specific position in the list
# it checks if the list is empty or if the index is out of range, and prints an error message if so.
# otherwise, it removes the element at the specified index using the pop method.
def delete_at_position(list, index):
    if len(list) == 0:
        print("List is empty, cannot delete")
    elif index < 0 or index >= len(list):
        print("Invalid position, cannot delete")
    else:
        list.pop(index)

# delete the first occurrence of a number in the list
def delete_number(list, num):
    if num in list:
        list.remove(num)
        print(f"{num} has been successfully deleted.")
    else:
        print(f"{num} is not on the list and cannot be deleted.")

# search for a number in the list and return its position
```

```python
def search_number(list, num):
    if num in list:
        print(f"Value {num} found at position {list.index(num)}")
    else:
        print(f"Value {num} not found")

# display the number at a specific position in the list
# if the index is less than 0 or greater than or equal to the length of the list,
it prints "Invalid position".
# otherwise, it prints the number at the specified position in the list.
def display_number_at_position(list, index):
    if index < 0 or index >= len(list):
        print("Invalid position")
    else:
        print(f"Number at position {index} is {list[index]}")

# display the entire list
def display_list(list):
    print(*list)

# main program
# this is a loop that runs until the user chooses to exit
# this is also incorporated with an error handling mechanism using the try-except
block to ensure that the user enters a valid choice

def main():
    list = [] # create an empty list
    while True:
        print("\nMenu:")
        print("[0] Exit")
        print("[1] Insert At Start")
        print("[2] Insert At End")
        print("[3] Insert At Position")
        print("[4] Delete At Start")
        print("[5] Delete At End")
        print("[6] Delete At Position")
        print("[7] Delete Number")
        print("[8] Search Number")
        print("[9] Display Number At Position")
        print("[10] Display List")
        try:
            choice = int(input("Enter your choice: "))
        except ValueError:
            print("Invalid choice. Please enter a number.")
            continue
        if choice == 0:
            confirm = input("Are you sure you want to exit? (y/n): ")
            if confirm.lower() == "y":
                print("Thank you!")
                break
        elif choice == 1:
            try:
```

```
                value = int(input("Enter the value to insert: "))
            except ValueError:
                print("Invalid value. Please enter a number.")
                continue
            insert_at_start(list, value)
            print(f"{value} has been successfully inserted at the start.")
        elif choice == 2:
            try:
                value = int(input("Enter the value to insert: "))
            except ValueError:
                print("Invalid value. Please enter a number.")
                continue
            insert_at_end(list, value)
            print(f"{value} has been successfully inserted at the end.")
        elif choice == 3:
            try:
                index = int(input("Enter the position to insert: "))
                value = int(input("Enter the value to insert: "))
            except ValueError:
                print("Invalid value. Please enter a number.")
                continue
            insert_at_position(list, index, value)
            print(f"{value} has been successfully inserted at position {index}.")
        elif choice == 4:
            delete_at_start(list)
            print(f"The value at the start has been successfully deleted.")
        elif choice == 5:
            delete_at_end(list)
            print(f"The value at the end has been successfully deleted.")
        elif choice == 6:
            try:
                index = int(input("Enter the position to delete: "))
            except ValueError:
                print("Invalid position. Please enter a number.")
                continue
            delete_at_position(list, index)
            print(f"The value at position {index} has been successfully
deleted.")
        elif choice == 7:
            try:
                num = int(input("Enter the number to delete: "))
            except ValueError:
                print("Invalid number. Please enter a number.")
                continue
            delete_number(list, num)
        elif choice == 8:
            try:
                num = int(input("Enter the number to search: "))
            except ValueError:
                print("Invalid number. Please enter a number.")
                continue
            search_number(list, num)
```

```
        elif choice == 9:
            try:
                index = int(input("Enter the position to display: "))
            except ValueError:
                print("Invalid position. Please enter a number.")
                continue
            display_number_at_position(list, index)
        elif choice == 10:
            if len(list) == 0:
                print("There are no numbers in the list.")
            else:
                display_list(list)
        else:
            print("Invalid choice")

if __name__ == "__main__":
    main()
```

TUTORIAL VIDEO

YouTube Link: https://youtu.be/nVHTWH6F8YM

TAKEAWAYS

Name of Member: ***ORIAS, MARK JOSEPH C.***
Contribution ~~to the Group~~: ***Part 1: Python Program File*** ***Part 2: Program Documentation*** ***Part 3: Tutorial Video***
Learnings: Throughout the discussion, I learned that list linear data structure are fundamental to data organization and manipulation in computer science. I learned that lists can be implemented in two primary ways: array-based and linked list-based. Array-based lists store elements in contiguous memory locations, allowing for efficient random access but costly insertions and deletions due to the need to shift elements. In contrast, linked lists consist of nodes that are dynamically allocated and linked together, providing flexibility for insertions and deletions without needing to shift elements but requiring linear time to access a specific element. Additionally, choosing the right data structure based on the specific requirements of a problem is important, as linked lists offer advantages in scenarios where dynamic memory allocation and frequent insertions/deletions are required, whereas arrays are more efficient for static data sets and scenarios requiring fast access. From the experience of coding and working with the list operations in Python, I have learned several key concepts and practices related to data structures and programming. First, I gained a deeper understanding of how lists can be manipulated using various functions, such as inserting and deleting elements at specific positions, searching for elements, and displaying elements at particular positions. This exercise highlighted the importance of handling edge cases, such as attempting to delete from

an empty list or accessing an index that is out of range, which reinforced my understanding of error handling and the robustness required in writing reliable code.

Furthermore, this emphasized the efficiency considerations involved in list operations. I realized that operations like insertion and deletion at the start or in the middle of a list are more time-consuming compared to operations at the end of the list due to the need to shift elements. This understanding aligns with the theoretical knowledge about the time complexity of different operations in arrays and lists. The practical experience of implementing these operations also improved my ability to write modular code, as I created specific functions for each operation, enhancing code reusability and readability.